

\$\$\$ sql.banco

DROP TABLE IF EXISTS Pecas;

DROP TABLE IF EXISTS sinistros;

DROP TABLE IF EXISTS Seguradoras;

```
CREATE TABLE Seguradoras (  
    nome VARCHAR(50) PRIMARY KEY,  
    cidade VARCHAR(50),  
    cobertura_percentual INT,  
    possui_atendimento_24h BOOLEAN,  
    forma_pagamento_preferencial VARCHAR(20)  
);
```

```
CREATE TABLE sinistros (  
    numero INT PRIMARY KEY,  
    nome VARCHAR(50),  
    telefone VARCHAR(20),  
    grau_monta VARCHAR(20),  
    perda_total BOOLEAN  
);
```

```
CREATE TABLE Pecas (  
    codigo INT PRIMARY KEY,  
    sinistro_numero INT,  
    nome VARCHAR(50),  
    categoria VARCHAR(20),  
    preco DECIMAL(10,2),  
    tipo VARCHAR(20),  
    cor VARCHAR(20),  
    mao_obra_propria BOOLEAN,  
    FOREIGN KEY (sinistro_numero) REFERENCES sinistros(numero)  
);
```

\$\$\$ src.controles.ControladorCadastroPecas

```
package controles;

import entidades.Pecas;
import interfaces.JanelaCadastroPecas;
import java.awt.Frame;

public class ControladorCadastroPecas {

    public ControladorCadastroPecas() {
        this(null);
    }

    public ControladorCadastroPecas(Frame owner) {
        JanelaCadastroPecas janela = new JanelaCadastroPecas(this, owner);
        janela.setVisible(true);
        janelaToFront();
        janela.requestFocus();
    }

    public String inserirPecas(Pecas pecas) {
        Pecas peca_buscada = Pecas.buscarPecas(pecas.getNome());
        if (peca_buscada == null) return Pecas.inserirPecas(pecas);
        else return "Nome de Peça já cadastrado";
    }

    public String alterarPecas(Pecas pecas) {
        Pecas peca_buscada = Pecas.buscarPecas(pecas.getNome());
        if (peca_buscada != null) return Pecas.alterarPecas(pecas);
        else return "Nome de Peça não cadastrado";
    }

    public String removerPecas(String nome) {
        Pecas peca_buscada = Pecas.buscarPecas(nome);
        if (peca_buscada != null) return Pecas.removerPecas(nome);
        else return "Nome de Peça não cadastrado";
    }
}
```

}

}

\$\$\$ src.controles.ControladorCadastroSeguradoras

```
package controles;

import entidades.Seguradora;
import interfaces.JanelaCadastroSeguradoras;
import java.awt.Frame;

public class ControladorCadastroSeguradoras {

    public ControladorCadastroSeguradoras() {
        this(null);
    }

    public ControladorCadastroSeguradoras(Frame owner) {
        JanelaCadastroSeguradoras janela = new JanelaCadastroSeguradoras(this, owner);
        janela.setVisible(true);
        janelaToFront();
        janela.requestFocus();
    }

    public String inserirSeguradora(Seguradora seguradora) {
        Seguradora seguradora_buscada = Seguradora.buscarSeguradora(seguradora.getNome());
        if (seguradora_buscada == null) return Seguradora.inserirSeguradora(seguradora);
        else return "Nome de Seguradora já cadastrado";
    }

    public String alterarSeguradora(Seguradora seguradora) {
        Seguradora seguradora_buscada = Seguradora.buscarSeguradora(seguradora.getNome());
        if (seguradora_buscada != null) return Seguradora.alterarSeguradora(seguradora);
        else return "Nome de Seguradora não cadastrado";
    }

    public String removerSeguradora(String nome) {
        Seguradora seguradora_buscada = Seguradora.buscarSeguradora(nome);
        if (seguradora_buscada != null) return Seguradora.removerSeguradora(nome);
        else return "Nome de Seguradora não cadastrado";
    }
}
```

}

}

\$\$\$ src.controles.ControladorCadastroSinistros

```
package controles;

import entidades.Sinistro;
import interfaces.JanelaCadastroSinistros;

public class ControladorCadastroSinistros {

    public ControladorCadastroSinistros() {
        new JanelaCadastroSinistros(this).setVisible(true);
    }

    public String inserirSinistro(Sinistro sinistro) {
        Sinistro sinistro_buscado = Sinistro.buscarSinistro(sinistro.getSequencial());
        if (sinistro_buscado == null) return Sinistro.inserirSinistro(sinistro);
        else return "Numero de Sinistro já cadastrado";
    }

    public String alterarSinistro(Sinistro sinistro) {
        Sinistro sinistro1 = Sinistro.buscarSinistro(sinistro.getSequencial());
        if (sinistro1 != null) return Sinistro.alterarSinistro(sinistro);
        else return "Numero de Sinistro não cadastrado";
    }

    public String removerSinistro(int sequencial) {
        Sinistro sinistro1 = Sinistro.buscarSinistro(sequencial);
        if (sinistro1 != null) return Sinistro.removerSinistro(sequencial);
        else return "Numero de Sinistro não cadastrado";
    }
}
```

\$\$\$ src.entidades.Pecas

```
package entidades;

import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.Locale;
import persistência.BD;

public class Pecas {

    public enum CategoriaPeca {
        OEM("OEM"),
        ORIGINAL("Original"),
        GENUINA("Genuína");

        private final String texto;

        CategoriaPeca(String texto) {
            this.texto = texto;
        }

        public static CategoriaPeca fromTexto(String texto) {
            if (texto == null) {
                throw new IllegalArgumentException("Categoria da peça não informada");
            }

            String normalizado = texto.trim().toLowerCase(Locale.ROOT);

            if (normalizado.equals("oem")) return OEM;
            if (normalizado.equals("original")) return ORIGINAL;
            if (normalizado.equals("genuina") || normalizado.equals("genuína")) return GENUINA;

            throw new IllegalArgumentException("Categoria da peça inválida: " + texto);
        }
    }
}
```

```
@Override
public String toString() {
    return texto;
}
}
```

```
public enum TipoPeca {
    MOTOR("motor"),
    INTERNO("interno"),
    EXTERNO("externo"),
    SUSPENSAO("suspensão");
```

```
    private final String texto;
```

```
    TipoPeca(String texto) {
        this.texto = texto;
    }
}
```

```
public static TipoPeca fromTexto(String texto) {
    if (texto == null) {
        throw new IllegalArgumentException("Tipo da peça não informado");
    }
}
```

```
    String normalizado = texto.trim().toLowerCase(Locale.ROOT);
```

```
    if (normalizado.equals("motor")) return MOTOR;
    if (normalizado.equals("interno")) return INTERNO;
    if (normalizado.equals("externo")) return EXTERNO;
    if (normalizado.equals("suspensao") || normalizado.equals("suspensão")) return
SUSPENSAO;
```

```
    throw new IllegalArgumentException("Tipo da peça inválido: " + texto);
}
```

```
@Override
public String toString() {
```

```
        return texto;
    }
}
```

```
private int codigo;
private String nome;
private CategoriaPeca categoria;
private double preco;
private TipoPeca tipo;
private String cor;
private boolean mao_obra_propria;
```

```
public int getCodigo() {
    return codigo;
}
```

```
public String getNome() {
    return nome;
}
```

```
public CategoriaPeca getCategoria() {
    return categoria;
}
```

```
public double getPreco() {
    return preco;
}
```

```
public TipoPeca getTipo() {
    return tipo;
}
```

```
public String getCor() {
    return cor;
}
```

```
public boolean getMaoObraPropria() {
```

```
    return mao_obra_propria;
}
```

```
public boolean isMaoObraPropria() {
    return mao_obra_propria;
}
```

```
public boolean getMaoDeObra() {
    return mao_obra_propria;
}
```

```
public void setCodigo(int codigo) {
    this.codigo = codigo;
}
```

```
public void setNome(String nome) {
    this.nome = nome;
}
```

```
public void setCategoria(CategoriaPeca categoria) {
    this.categoria = categoria;
}
```

```
public void setPreco(double preco) {
    this.preco = preco;
}
```

```
public void setTipo(TipoPeca tipo) {
    this.tipo = tipo;
}
```

```
public void setCor(String cor) {
    this.cor = cor;
}
```

```
public void setMaoObraPropria(boolean mao_obra_propria) {
    this.mao_obra_propria = mao_obra_propria;
}
```

```
}
```

```
public void setMaoDeObra(boolean mao_de_obra) {  
    this.mao_obra_propria = mao_de_obra;  
}
```

```
public Pecas(int codigo, String nome, CategoriaPeca categoria,  
             double preco, TipoPeca tipo, String cor,  
             boolean mao_obra_propria) {  
  
    this.codigo = codigo;  
    this.nome = nome;  
    this.categoria = categoria;  
    this.preco = preco;  
    this.tipo = tipo;  
    this.cor = cor;  
    this.mao_obra_propria = mao_obra_propria;  
}
```

```
public String toString() {  
    return codigo + " - " + nome + " [" + categoria + "];"  
}
```

```
public static Pecas[] getVisoes() {
```

```
    String sql = "SELECT Codigo, Nome, Categoria FROM Pecas";
```

```
    ResultSet lista_resultados = null;
```

```
    ArrayList<Pecas> visoes = new ArrayList();
```

```
    try {
```

```
        PreparedStatement comando =
```

```
            BD.conexao.prepareStatement(sql);
```

```
        lista_resultados = comando.executeQuery();
```

```

while (lista_resultados.next()) {

    try {
        int codigo =
            lista_resultados.getInt("Codigo");

        String nome =
            lista_resultados.getString("Nome");

        String categoria =
            lista_resultados.getString("Categoria");

        visoes.add(new Pecas(
            codigo,
            nome,
            CategoriaPeca.fromTexto(categoria),
            0,
            null,
            null,
            false
        ));
    } catch (IllegalArgumentException excecao_enum) {
        // Ignora registros com valor fora do domínio esperado.
    }
}

lista_resultados.close();
comando.close();

} catch (SQLException excecao_sql) {
    excecao_sql.printStackTrace();
}

return visoes.toArray(new Pecas[visoes.size()]);
}

public Pecas getVisao() {

```

```
return new Pecas(  
    codigo,  
    nome,  
    categoria,  
    preco,  
    tipo,  
    cor,  
    mao_obra_propria  
);  
}
```

```
public static Pecas buscarPecas(String nome) {
```

```
    String sql =  
        "SELECT * FROM Pecas WHERE Nome = ?";
```

```
    ResultSet lista_resultados = null;  
    Pecas pecas = null;
```

```
    try {
```

```
        PreparedStatement comando =  
            BD.conexao.prepareStatement(sql);
```

```
        comando.setString(1, nome);
```

```
        lista_resultados = comando.executeQuery();
```

```
        while (lista_resultados.next()) {
```

```
            try {  
                pecas = new Pecas(  
                    lista_resultados.getInt("Codigo"),  
                    lista_resultados.getString("Nome"),  
                    CategoriaPeca.fromTexto(  
                        lista_resultados.getString("Categoria")  
                    ),  
                ),  
            }  
        }
```

```

        lista_resultados.getDouble("Preco"),
        TipoPeca.fromTexto(lista_resultados.getString("Tipo")),
        lista_resultados.getString("Cor"),
        lista_resultados.getBoolean("MaoObraPropria")
    );
} catch (IllegalArgumentException excecao_enum) {
    pecas = null;
}
}

lista_resultados.close();
comando.close();

} catch (SQLException excecao_sql) {

    excecao_sql.printStackTrace();
    pecas = null;
}

return pecas;
}

public static String inserirPecas(Pecas pecas) {

    String sql =
        "INSERT INTO Pecas " +
        "(Codigo, Nome, Categoria, Preco, Tipo, Cor, MaoObraPropria) " +
        "VALUES (?,?,?,?,?,?,?)";

    try {

        PreparedStatement comando =
            BD.conexao.prepareStatement(sql);

        comando.setInt(1, pecas.getCodigo());
        comando.setString(2, pecas.getNome());
        comando.setString(3, pecas.getCategoria().toString());
    }
}

```

```

        comando.setDouble(4, pecas.getPreco());
        comando.setString(5, pecas.getTipo().toString());
        comando.setString(6, pecas.getCor());
        comando.setBoolean(7, pecas.getMaoObraPropria());

        comando.executeUpdate();
        comando.close();

        return null;

    } catch (SQLException excecao_sql) {

        excecao_sql.printStackTrace();

        return "Erro na Inserção da Peça no BD";
    }
}

```

```

public static String alterarPecas(Pecas pecas) {

```

```

    String sql =
        "UPDATE Pecas SET " +
        "Codigo = ?, " +
        "Categoria = ?, " +
        "Preco = ?, " +
        "Tipo = ?, " +
        "Cor = ?, " +
        "MaoObraPropria = ? " +
        "WHERE Nome = ?";

```

```

try {

```

```

    PreparedStatement comando =
        BD.conexao.prepareStatement(sql);

```

```

    comando.setInt(1, pecas.getCodigo());
    comando.setString(2, pecas.getCategoria().toString());

```

```

        comando.setDouble(3, pecas.getPreco());
        comando.setString(4, pecas.getTipo().toString());
        comando.setString(5, pecas.getCor());
        comando.setBoolean(6, pecas.getMaoObraPropria());
        comando.setString(7, pecas.getNome());

        comando.executeUpdate();
        comando.close();

        return null;

    } catch (SQLException excecao_sql) {

        excecao_sql.printStackTrace();

        return "Erro na Alteração da Peça no BD";
    }
}

```

```

public static String removerPecas(String nome) {

    String sql =
        "DELETE FROM Pecas WHERE Nome = ?";

    try {

        PreparedStatement comando =
            BD.conexao.prepareStatement(sql);

        comando.setString(1, nome);

        comando.executeUpdate();
        comando.close();

        return null;

    } catch (SQLException excecao_sql) {

```

```
excecao_sql.printStackTrace();
```

```
return "Erro na Remoção da Peça no BD";
```

```
}
```

```
}
```

```
}
```

\$\$\$ src.entidades.Seguradora

```
package entidades;

import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.Locale;
import persistência.BD;

public class Seguradora {

    public enum FormaPagamentoPreferencial {
        BOLETO("boleto"),
        CARTAO("cartao"),
        PIX("pix"),
        DEBITO_AUTOMATICO("debito_automatico");

        private final String texto;

        FormaPagamentoPreferencial(String texto) {
            this.texto = texto;
        }

        public static FormaPagamentoPreferencial fromTexto(String texto) {
            if (texto == null) {
                throw new IllegalArgumentException("Forma de pagamento não informada");
            }

            String normalizado = texto.trim().toLowerCase(Locale.ROOT);

            if (normalizado.equals("boleto")) return BOLETO;
            if (normalizado.equals("cartao") || normalizado.equals("cartão")) return CARTAO;
            if (normalizado.equals("pix")) return PIX;
            if (normalizado.equals("debito_automatico")
                || normalizado.equals("debito automatico"))
```

```

        || normalizado.equals("débito_automático")
        || normalizado.equals("débito automático")) {
    return DEBITO_AUTOMATICO;
}

```

```

    throw new IllegalArgumentException("Forma de pagamento inválida: " + texto);
}

```

```

@Override
public String toString() {
    return texto;
}
}

```

```

private String nome;
private String cidade;
private double cobertura_percentual;
private boolean possui_atendimento_24h;
private FormaPagamentoPreferencial forma_pagamento_preferencial;

```

```

public String getNome() { return nome; }
public String getCidade() { return cidade; }
public double getCoberturaPercentual() { return cobertura_percentual; }
public boolean getPossuiAtendimento24h() { return possui_atendimento_24h; }
public FormaPagamentoPreferencial getFormaPagamentoPreferencial() { return
forma_pagamento_preferencial; }

```

```

public void setCidade(String cidade) { this.cidade = cidade; }
public void setCoberturaPercentual(double cobertura_percentual) { this.cobertura_percentual =
cobertura_percentual; }
public void setPossuiAtendimento24h(boolean possui_atendimento_24h)
{ this.possui_atendimento_24h = possui_atendimento_24h; }
public void setFormaPagamentoPreferencial(FormaPagamentoPreferencial
forma_pagamento_preferencial) {
    this.forma_pagamento_preferencial = forma_pagamento_preferencial;
}

```

```

public Seguradora(

```

```

        String nome,
        String cidade,
        double cobertura_percentual,
        boolean possui_atendimento_24h,
        FormaPagamentoPreferencial forma_pagamento_preferencial
    ) {
        this.nome = nome;
        this.cidade = cidade;
        this.cobertura_percentual = cobertura_percentual;
        this.possui_atendimento_24h = possui_atendimento_24h;
        this.forma_pagamento_preferencial = forma_pagamento_preferencial;
    }

    public Seguradora(String nome, String cidade, double cobertura_percentual) {
        this(nome, cidade, cobertura_percentual, false, FormaPagamentoPreferencial.BOLETO);
    }

    public Seguradora(String nome, String cidade) {
        this(nome, cidade, 0, false, FormaPagamentoPreferencial.BOLETO);
    }

    public String toString() {
        return nome + " [" + cidade + "]";
    }

    public Seguradora getVisao() {
        return new Seguradora(
            nome,
            cidade,
            cobertura_percentual,
            possui_atendimento_24h,
            forma_pagamento_preferencial
        );
    }

    public static Seguradora[] getVisoes() {
        String sql = "SELECT nome, cidade, cobertura_percentual, possui_atendimento_24h,

```

```

forma_pagamento_preferencial FROM Seguradoras";

ResultSet lista_resultados = null;
ArrayList<Seguradora> visoes = new ArrayList<>();

try {
    PreparedStatement comando = BD.conexao.prepareStatement(sql);
    lista_resultados = comando.executeQuery();

    while (lista_resultados.next()) {
        String nome = lista_resultados.getString("nome");
        String cidade = lista_resultados.getString("cidade");
        double cobertura_percentual = lista_resultados.getDouble("cobertura_percentual");
        boolean possui_atendimento_24h =
lista_resultados.getBoolean("possui_atendimento_24h");
        FormaPagamentoPreferencial forma_pagamento_preferencial =
            FormaPagamentoPreferencial.fromTexto(
                lista_resultados.getString("forma_pagamento_preferencial")
            );
        visoes.add(new Seguradora(
            nome,
            cidade,
            cobertura_percentual,
            possui_atendimento_24h,
            forma_pagamento_preferencial
        ));
    }

    lista_resultados.close();
    comando.close();

} catch (SQLException excecao_sql) {
    excecao_sql.printStackTrace();
}

return visoes.toArray(new Seguradora[visoes.size()]);
}

```

```

public static Seguradora buscarSeguradora(String nome) {
    String sql = "SELECT * FROM Seguradoras WHERE nome = ?";
    ResultSet lista_resultados = null;
    Seguradora seguradora = null;

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        comando.setString(1, nome);
        lista_resultados = comando.executeQuery();

        while (lista_resultados.next()) {
            seguradora = new Seguradora(
                nome,
                lista_resultados.getString("cidade"),
                lista_resultados.getDouble("cobertura_percentual"),
                lista_resultados.getBoolean("possui_atendimento_24h"),
                FormaPagamentoPreferencial.fromTexto(
                    lista_resultados.getString("forma_pagamento_preferencial")
                )
            );
        }

        lista_resultados.close();
        comando.close();

    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
        seguradora = null;
    }

    return seguradora;
}

```

```

public static String inserirSeguradora(Seguradora seguradora) {
    String sql = "INSERT INTO Seguradoras (nome, cidade, cobertura_percentual,
    possui_atendimento_24h, forma_pagamento_preferencial) VALUES (?, ?, ?, ?, ?)";
}

```

```

try {
    PreparedStatement comando = BD.conexao.prepareStatement(sql);

    comando.setString(1, seguradora.getNome());
    comando.setString(2, seguradora.getCidade());
    comando.setDouble(3, seguradora.getCoberturaPercentual());
    comando.setBoolean(4, seguradora.getPossuiAtendimento24h());
    comando.setString(5, seguradora.getFormaPagamentoPreferencial().toString());

    comando.executeUpdate();
    comando.close();

    return null;

} catch (SQLException excecao_sql) {
    excecao_sql.printStackTrace();
    return "Erro na Inserção da Seguradora no BD";
}
}

```

```

public static String alterarSeguradora(Seguradora seguradora) {
    String sql = "UPDATE Seguradoras SET cidade = ?, cobertura_percentual = ?,
possui_atendimento_24h = ?, forma_pagamento_preferencial = ?"
        + " WHERE nome = ?";

```

```

try {
    PreparedStatement comando = BD.conexao.prepareStatement(sql);

    comando.setString(1, seguradora.getCidade());
    comando.setDouble(2, seguradora.getCoberturaPercentual());
    comando.setBoolean(3, seguradora.getPossuiAtendimento24h());
    comando.setString(4, seguradora.getFormaPagamentoPreferencial().toString());
    comando.setString(5, seguradora.getNome());

    comando.executeUpdate();
    comando.close();

```

```
return null;
```

```
    } catch (SQLException excecao_sql) {  
        excecao_sql.printStackTrace();  
        return "Erro na Alteração da Seguradora no BD";  
    }  
}
```

```
public static String removerSeguradora(String nome) {  
    String sql = "DELETE FROM Seguradoras WHERE nome = ?";
```

```
    try {  
        PreparedStatement comando = BD.conexao.prepareStatement(sql);  
  
        comando.setString(1, nome);  
        comando.executeUpdate();  
        comando.close();
```

```
    return null;
```

```
    } catch (SQLException excecao_sql) {  
        excecao_sql.printStackTrace();  
        return "Erro na Remoção da Seguradora no BD";  
    }
```

```
    }  
}
```

\$\$\$ src.entidades.Sinistro

```
package entidades;

import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.Locale;
import persistência.BD;

public class Sinistro {

    public enum GrauMonta {
        PEQUENA("pequena"),
        MEDIA("media"),
        GRANDE("grande");

    }

    private final String texto;

    GrauMonta(String texto) {
        this.texto = texto;
    }

    public static GrauMonta fromTexto(String texto) {
        if (texto == null) {
            throw new IllegalArgumentException("Grau de monta não informado");
        }

        String normalizado = texto.trim().toLowerCase(Locale.ROOT);
        normalizado = normalizado
            .replace("á", "a")
            .replace("à", "a")
            .replace("â", "a")
            .replace("ã", "a")
            .replace("é", "e")
            .replace("ê", "e")
    }
}
```

```
.replace("í", "i")
.replace("ó", "o")
.replace("ô", "o")
.replace("ø", "o")
.replace("ú", "u")
.replace("ç", "c");
```

```
if (normalizado.contains("pequena")) return PEQUENA;
if (normalizado.contains("media")) return MEDIA;
if (normalizado.contains("grande")) return GRANDE;
```

```
throw new IllegalArgumentException("Grau de monta inválido: " + texto);
}
```

```
@Override
public String toString() {
    return texto;
}
}
```

```
private int numero;
private String cliente;
private String telefone;
private GrauMonta grau_monta;
private boolean perda_total;
```

```
public int getNumero() { return numero; }
public String getCliente() { return cliente; }
public String getTelefone() { return telefone; }
public GrauMonta getGrauMonta() { return grau_monta; }
public boolean getPerdaTotal() { return perda_total; }
public boolean isPerdaTotal() { return perda_total; }
```

```
public void setNumero(int numero) { this.numero = numero; }
public void setCliente(String cliente) { this.cliente = cliente; }
public void setTelefone(String telefone) { this.telefone = telefone; }
public void setGrauMonta(GrauMonta grau_monta) {
```

```

        this.grau_monta = grau_monta != null ? grau_monta : GrauMonta.PEQUENA;
    }
    public void setPerdaTotal(boolean perda_total) { this.perda_total = perda_total; }

    public Sinistro(
        int numero,
        String cliente,
        String telefone,
        GrauMonta grau_monta,
        boolean perda_total
    ) {
        this.numero = numero;
        this.cliente = cliente;
        this.telefone = telefone;
        this.grau_monta = grau_monta != null ? grau_monta : GrauMonta.PEQUENA;
        this.perda_total = perda_total;
    }

    public Sinistro(int numero, String cliente, String telefone) {
        this(numero, cliente, telefone, GrauMonta.PEQUENA, false);
    }

    public Sinistro(String cliente, String telefone) {
        this(0, cliente, telefone, GrauMonta.PEQUENA, false);
    }

    public Sinistro(String cliente, String telefone, GrauMonta grau_monta, boolean perda_total) {
        this(0, cliente, telefone, grau_monta, perda_total);
    }

    public int getSequencial() { return numero; }
    public void setSequencial(int sequencial) { this.numero = sequencial; }

    @Override
    public String toString() {
        return "[" + numero + "] " + cliente + " (" + grau_monta + ")";
    }

```

```
public Sinistro getVisao() {  
    return new Sinistro(numero, cliente, telefone, grau_monta, perda_total);  
}
```

```
public static Sinistro[] getVisoes() {  
    String sql = "SELECT numero, nome, telefone, grau_monta, perda_total FROM sinistros";  
    ResultSet lista_resultados = null;  
    ArrayList<Sinistro> visoes = new ArrayList<>();
```

```
    try {  
        PreparedStatement comando = BD.conexao.prepareStatement(sql);  
        lista_resultados = comando.executeQuery();  
  
        while (lista_resultados.next()) {  
            try {  
                int numero = lista_resultados.getInt("numero");  
                String cliente = lista_resultados.getString("nome");  
                String telefone = lista_resultados.getString("telefone");  
                GrauMonta grau_monta = GrauMonta.fromTexto(  
                    lista_resultados.getString("grau_monta")  
                );  
                boolean perda_total = lista_resultados.getBoolean("perda_total");  
                visoes.add(new Sinistro(  
                    numero,  
                    cliente,  
                    telefone,  
                    grau_monta,  
                    perda_total  
                ));  
            } catch (IllegalArgumentException excecao_enum) {  
                // Ignora registros com valor fora do domínio esperado.  
            }  
        }  
    }
```

```
    lista_resultados.close();  
    comando.close();
```

```

    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
    }

    return visoes.toArray(new Sinistro[visoes.size()]);
}

public static Sinistro buscarSinistro(int numero) {
    String sql = "SELECT * FROM sinistros WHERE numero = ?";
    ResultSet lista_resultados = null;
    Sinistro sinistro = null;

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        comando.setInt(1, numero);
        lista_resultados = comando.executeQuery();

        while (lista_resultados.next()) {
            try {
                sinistro = new Sinistro(
                    numero,
                    lista_resultados.getString("nome"),
                    lista_resultados.getString("telefone"),
                    GrauMonta.fromTexto(lista_resultados.getString("grau_monta")),
                    lista_resultados.getBoolean("perda_total")
                );
            } catch (IllegalArgumentException excecao_enum) {
                sinistro = null;
            }
        }

        lista_resultados.close();
        comando.close();
    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
        sinistro = null;
    }
}

```

```
    return sinistro;
}
```

```
public static String inserirSinistro(Sinistro sinistro) {
    String sql = "INSERT INTO sinistros (numero, nome, telefone, grau_monta, perda_total)
VALUES (?, ?, ?, ?, ?)";
```

```
    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        comando.setInt(1, sinistro.getNumero());
        comando.setString(2, sinistro.getCliente());
        comando.setString(3, sinistro.getTelefone());
        comando.setString(4, sinistro.getGrauMonta().toString());
        comando.setBoolean(5, sinistro.getPerdaTotal());
        comando.executeUpdate();
        comando.close();
        return null;
    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
        return "Erro na Inserção do Sinistro no BD";
    }
}
```

```
public static String alterarSinistro(Sinistro sinistro) {
    String sql = "UPDATE sinistros SET nome = ?, telefone = ?, grau_monta = ?, perda_total = ?
WHERE numero = ?";
```

```
    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        comando.setString(1, sinistro.getCliente());
        comando.setString(2, sinistro.getTelefone());
        comando.setString(3, sinistro.getGrauMonta().toString());
        comando.setBoolean(4, sinistro.getPerdaTotal());
        comando.setInt(5, sinistro.getNumero());
        comando.executeUpdate();
        comando.close();
```

```

        return null;
    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
        return "Erro na Alteração do Sinistro no BD";
    }
}

```

```

public static String removerSinistro(int numero) {
    String sql = "DELETE FROM sinistros WHERE numero = ?";

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        comando.setInt(1, numero);
        comando.executeUpdate();
        comando.close();
        return null;
    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
        return "Erro na Remoção do Sinistro no BD";
    }
}

```

```

public static int ultimoNumero() {
    String sql = "SELECT MAX(numero) FROM sinistros";
    ResultSet lista_resultados = null;
    int numero = 0;

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        lista_resultados = comando.executeQuery();

        while (lista_resultados.next()) {
            numero = lista_resultados.getInt(1);
        }

        lista_resultados.close();
        comando.close();
    }
}

```

```

    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
    }

    return numero;
}

public static boolean existeSinistroMesmosAtributos(Sinistro sinistro) {
    String sql = "SELECT COUNT(numero) FROM sinistros WHERE nome = ? AND telefone
= ? AND grau_monta = ? AND perda_total = ?";
    ResultSet lista_resultados = null;
    int n_sinistros_mesmos_atributos = 0;

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        comando.setString(1, sinistro.getCliente());
        comando.setString(2, sinistro.getTelefone());
        comando.setString(3, sinistro.getGrauMonta().toString());
        comando.setBoolean(4, sinistro.getPerdaTotal());
        lista_resultados = comando.executeQuery();

        while (lista_resultados.next()) {
            n_sinistros_mesmos_atributos = lista_resultados.getInt(1);
        }

        lista_resultados.close();
        comando.close();
    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
    }

    return n_sinistros_mesmos_atributos > 0;
}
}

```

\$\$\$ src.interfaces.JanelaCadastroPecas

```
package interfaces;

import javax.swing.JOptionPane;
import controles.ControladorCadastroPecas;
import entidades.Pecas;
import javax.swing.DefaultComboBoxModel;
import java.awt.Frame;

public class JanelaCadastroPecas extends javax.swing.JFrame {

    ControladorCadastroPecas controlador;
    Pecas[] pecas_cadastradas;

    public JanelaCadastroPecas(ControladorCadastroPecas controlador) {
        this(controlador, null);
    }

    public JanelaCadastroPecas(
        ControladorCadastroPecas controlador,
        Frame owner
    ) {
        this.controlador = controlador;
        pecas_cadastradas = Pecas.getVisoes();
        initComponents();
        atualizarPecasCadastradas();
        setSize(new java.awt.Dimension(720, 560));
        configurarJanelaDependente(owner);
        limparCampos(null);
    }

    private void configurarJanelaDependente(Frame owner) {
        setAutoRequestFocus(true);
        setAlwaysOnTop(true);

        if (owner == null) {
```

```
        setLocationByPlatform(true);  
        return;  
    }
```

```
    posicionarRelativaAoOwner(owner);  
}
```

```
private void posicionarRelativaAoOwner(Frame owner) {  
    int x = owner.getX() + (owner.getWidth() - getWidth()) / 2;  
    int y = owner.getY() + (owner.getHeight() - getHeight()) / 2;  
    setLocation(x, y);  
}
```

```
private Pecas localizarPeca(int codigo) {  
    for (Pecas visao : pecas_cadastradas) {  
        if (visao.getCodigo() == codigo) return visao;  
    }  
    return null;  
}
```

```
private void atualizarPecasCadastradas() {  
    atualizarPecasCadastradas(-1);  
}
```

```
private void atualizarPecasCadastradas(int codigo_selecionado) {  
    pecas_cadastradas = Pecas.getVisoes();  
    pecas_cadastradasComboBox.setModel(  
        new DefaultComboBoxModel(pecas_cadastradas)  
    );  
  
    Pecas visao_selecionada = localizarPeca(codigo_selecionado);  
    if (visao_selecionada != null) {  
        pecas_cadastradasComboBox.setSelectedItem(visao_selecionada);  
    }  
}
```

```
private void informarErro(String mensagem) {
```

```
JOptionPane.showMessageDialog(  
    this,  
    mensagem,  
    "Erro",  
    JOptionPane.ERROR_MESSAGE  
);  
}
```

```
private Pecas obterPecasInformada() {  
    String codigoStr = codigoTextField.getText();  
    if (codigoStr.isEmpty()) return null;
```

```
    int codigo;  
    try {  
        codigo = Integer.parseInt(codigoStr);  
    } catch (NumberFormatException e) {  
        return null;  
    }
```

```
    String nome = nomeTextField.getText();  
    if (nome.isEmpty()) return null;
```

```
    Pecas.CategoriaPeca categoria =  
        (Pecas.CategoriaPeca) categoriaComboBox.getSelectedItem();  
    if (categoria == null) return null;
```

```
    String precoStr = precoTextField.getText();  
    if (precoStr.isEmpty()) return null;
```

```
    double preco;  
    try {  
        preco = Double.parseDouble(precoStr.replace(",", "."));  
    } catch (NumberFormatException e) {  
        return null;  
    }
```

```
    String tipo = tipoTextField.getText();
```

```

if (tipo.isEmpty()) return null;

String cor = corTextField.getText();
if (cor.isEmpty()) return null;

boolean mao_de_obra = maoDeObraCheckBox.isSelected();

try {
    return new Pecas(
        codigo,
        nome,
        categoria,
        preco,
        Pecas.TipoPeca.fromTexto(tipo),
        cor,
        mao_de_obra
    );
} catch (IllegalArgumentException excecao) {
    return null;
}
}

private Pecas getVisaoAlterada(String nome) {
    for (Pecas visao : pecas_cadastradas) {
        if (visao.getNome().equals(nome)) return visao;
    }
    return null;
}

@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {
    java.awt.GridBagConstraints gridBagConstraints;

    comandosPanel = new javax.swing.JPanel();
    inserirPecas = new javax.swing.JButton();
    alterarPecas = new javax.swing.JButton();

```

```

consultarPecas = new javax.swing.JButton();
removerPecas = new javax.swing.JButton();
limparCampos = new javax.swing.JButton();
codigoLabel = new javax.swing.JLabel();
codigoTextField = new javax.swing.JTextField();
nomeLabel = new javax.swing.JLabel();
nomeTextField = new javax.swing.JTextField();
categoriaLabel = new javax.swing.JLabel();
categoriaTextField = new javax.swing.JTextField();
precoLabel = new javax.swing.JLabel();
precoTextField = new javax.swing.JTextField();
tipoLabel = new javax.swing.JLabel();
tipoTextField = new javax.swing.JTextField();
corLabel = new javax.swing.JLabel();
corTextField = new javax.swing.JTextField();
maoDeObraLabel = new javax.swing.JLabel();
maoDeObraTextField = new javax.swing.JTextField();
pecas_cadastradasComboBox = new javax.swing.JComboBox();
pecasCadastradasLabel = new javax.swing.JLabel();

setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
setTitle("Cadastrar Peças");
setMinimumSize(new java.awt.Dimension(640, 420));
setPreferredSize(new java.awt.Dimension(720, 560));
java.awt.GridBagLayout layout = new java.awt.GridBagLayout();
layout.rowHeights = new int[] {8};
getContentPane().setLayout(layout);

inserirPecas.setText("Inserir");
inserirPecas.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        inserirPecas(evt);
    }
});
comandosPanel.add(inserirPecas);

alterarPecas.setText("Alterar");

```

```
alterarPecas.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        alterarPecas(evt);  
    }  
});  
comandosPanel.add(alterarPecas);
```

```
consultarPecas.setText("Consultar");  
consultarPecas.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        consultarPecas(evt);  
    }  
});  
comandosPanel.add(consultarPecas);
```

```
removerPecas.setText("Remover");  
removerPecas.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        removerPecas(evt);  
    }  
});  
comandosPanel.add(removerPecas);
```

```
limparCampos.setText("Limpar");  
limparCampos.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        limparCampos(evt);  
    }  
});  
comandosPanel.add(limparCampos);
```

```
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 1;  
gridBagConstraints.gridy = 9;  
gridBagConstraints.insets = new java.awt.Insets(5, 5, 5, 5);  
getContentPane().add(comandosPanel, gridBagConstraints);
```

```
codigoLabel.setText("Código");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 1;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(codigoLabel, gridBagConstraints);
```

```
codigoTextField.setColumns(50);
codigoTextField.setPreferredSize(new java.awt.Dimension(80, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 1;
gridBagConstraints.ipadx = 399;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(codigoTextField, gridBagConstraints);
```

```
nomeLabel.setText("Nome");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 2;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(nomeLabel, gridBagConstraints);
```

```
nomeTextField.setColumns(50);
nomeTextField.setPreferredSize(new java.awt.Dimension(240, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 2;
gridBagConstraints.ipadx = 200;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(nomeTextField, gridBagConstraints);
```

```
categoriaLabel.setText("Categoria");
```

```
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 3;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(categoriaLabel, gridBagConstraints);
```

```
categoriaTextField.setColumns(50);
categoriaTextField.setPreferredSize(new java.awt.Dimension(180, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 3;
gridBagConstraints.ipadx = 100;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(categoriaTextField, gridBagConstraints);
```

```
precoLabel.setText("Preço");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 4;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(precoLabel, gridBagConstraints);
```

```
precoTextField.setColumns(50);
precoTextField.setPreferredSize(new java.awt.Dimension(100, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 4;
gridBagConstraints.ipadx = 18;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(precoTextField, gridBagConstraints);
```

```
tipoLabel.setText("Tipo");
gridBagConstraints = new java.awt.GridBagConstraints();
```

```
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 5;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(tipoLabel, gridBagConstraints);
```

```
tipoTextField.setColumns(14);
tipoTextField.setPreferredSize(new java.awt.Dimension(140, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 5;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(tipoTextField, gridBagConstraints);
```

```
corLabel.setText("Cor");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 6;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(corLabel, gridBagConstraints);
```

```
corTextField.setColumns(10);
corTextField.setPreferredSize(new java.awt.Dimension(100, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 6;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(corTextField, gridBagConstraints);
```

```
maoDeObraLabel.setText("Mão de Obra");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 7;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
```

```
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(maoDeObraLabel, gridBagConstraints);
```

```
maoDeObraTextField.setColumns(8);
maoDeObraTextField.setPreferredSize(new java.awt.Dimension(80, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 7;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(maoDeObraTextField, gridBagConstraints);
```

```
pecas_cadastradasComboBox.setModel(new DefaultComboBoxModel(pecas_cadastradas));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 0;
gridBagConstraints.anchor = java.awt.GridBagConstraints.NORTHWEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(pecas_cadastradasComboBox, gridBagConstraints);
```

```
pecasCadastradasLabel.setText("Peças Cadastradas");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 0;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(pecasCadastradasLabel, gridBagConstraints);
```

```
pack();
```

```
// </editor-fold>
```

```
private void inserirPecas(java.awt.event.ActionEvent evt) {
    Pecas pecas = obterPecasInformada();
    String mensagem_erro = null;

    if (pecas != null)
        mensagem_erro = controlador.inserirPecas(pecas);
}
```

```

else
    mensagem_erro = "Algum atributo da Peça não foi informado";

if (mensagem_erro == null) {
    Pecas visao = pecas.getVisao();
    atualizarPecasCadastradas(visao.getCodigo());
} else {
    informarErro(mensagem_erro);
}
}

private void alterarPecas(java.awt.event.ActionEvent evt) {
    Pecas pecas = obterPecasInformada();
    String mensagem_erro = null;

    if (pecas != null)
        mensagem_erro = controlador.alterarPecas(pecas);
    else
        mensagem_erro = "Algum atributo da Peça não foi informado";

    if (mensagem_erro == null) {
        Pecas visao = getVisaoAlterada(pecas.getNome());

        if (visao != null) {
            visao.setCodigo(pecas.getCodigo());
            visao.setCategoria(pecas.getCategoria());
            visao.setPreco(pecas.getPreco());
            visao.setTipo(pecas.getTipo());
            visao.setCor(pecas.getCor());
            visao.setMaoDeObra(pecas.getMaoDeObra());

            pecas_cadastradasComboBox.updateUI();
            atualizarPecasCadastradas(pecas.getCodigo());
        }
    } else {
        informarErro(mensagem_erro);
    }
}

```

```
}
```

```
private void consultarPecas(java.awt.event.ActionEvent evt) {  
    Pecas visao =  
        (Pecas) pecas_cadastradasComboBox.getSelectedItem();  
  
    Pecas pecas = null;  
    String mensagem_erro = null;  
  
    if (visao != null) {  
        pecas = Pecas.buscarPecas(visao.getNome());  
  
        if (pecas == null)  
            mensagem_erro = "Peça não cadastrada";  
    } else {  
        mensagem_erro = "Nenhuma Peça selecionada";  
    }  
  
    if (mensagem_erro == null) {  
  
        codigoTextField.setText(  
            String.valueOf(pecas.getCodigo())  
        );  
  
        nomeTextField.setText(pecas.getNome());  
        categoriaComboBox.setSelectedItem(pecas.getCategoria());  
  
        precoTextField.setText(  
            String.valueOf(pecas.getPreco())  
        );  
  
        tipoTextField.setText(pecas.getTipo().toString());  
        corTextField.setText(pecas.getCor());  
  
        maoDeObraCheckBox.setSelected(pecas.getMaoDeObra());  
  
    } else {
```

```

        informarErro(mensagem_erro);
    }
}

private void removerPecas(java.awt.event.ActionEvent evt) {
    Pecas visao =
        (Pecas) pecas_cadastradasComboBox.getSelectedItemAt();

    String mensagem_erro = null;

    if (visao != null)
        mensagem_erro =
            controlador.removerPecas(visao.getNome());
    else
        mensagem_erro = "Nenhuma Peça selecionada";

    if (mensagem_erro == null) {
        atualizarPecasCadastradas();
        limparCampos(evt);
    } else {
        informarErro(mensagem_erro);
    }
}

private void limparCampos(java.awt.event.ActionEvent evt) {
    codigoTextField.setText("");
    nomeTextField.setText("");
    categoriaComboBox.setSelectedIndex(0);
    precoTextField.setText("");
    tipoTextField.setText("");
    corTextField.setText("");
    maoDeObraCheckBox.setSelected(false);
}

// Variables declaration - do not modify
private javax.swing.JButton alterarPecas;
private javax.swing.JLabel categoriaLabel;

```

```
private javax.swing.JTextField categoriaTextField;
private javax.swing.JLabel codigoLabel;
private javax.swing.JTextField codigoTextField;
private javax.swing.JPanel comandosPanel;
private javax.swing.JButton consultarPecas;
private javax.swing.JLabel corLabel;
private javax.swing.JTextField corTextField;
private javax.swing.JButton inserirPecas;
private javax.swing.JButton limparCampos;
private javax.swing.JLabel maoDeObraLabel;
private javax.swing.JTextField maoDeObraTextField;
private javax.swing.JLabel nomeLabel;
private javax.swing.JTextField nomeTextField;
private javax.swing.JLabel pecasCadastradasLabel;
private javax.swing.JComboBox pecas_cadastradasComboBox;
private javax.swing.JLabel precoLabel;
private javax.swing.JTextField precoTextField;
private javax.swing.JButton removerPecas;
private javax.swing.JLabel tipoLabel;
private javax.swing.JTextField tipoTextField;
// End of variables declaration
}
```

\$\$\$ src.interfaces.JanelaCadastroSeguradoras

```
package interfaces;

import javax.swing.JOptionPane;
import controles.ControladorCadastroSeguradoras;
import entidades.Seguradora;
import javax.swing.DefaultComboBoxModel;
import java.awt.Frame;

public class JanelaCadastroSeguradoras extends javax.swing.JFrame {

    ControladorCadastroSeguradoras controlador;
    Seguradora[] seguradoras_cadastradas;

    public JanelaCadastroSeguradoras(ControladorCadastroSeguradoras controlador) {
        this(controlador, null);
    }

    public JanelaCadastroSeguradoras(
        ControladorCadastroSeguradoras controlador,
        Frame owner
    ) {
        this.controlador = controlador;
        seguradoras_cadastradas = Seguradora.getVisoes();
        initComponents();
        atualizarSeguradorasCadastradas();
        setSize(new java.awt.Dimension(760, 540));
        configurarJanelaDependente(owner);
        limparCampos(null);
    }

    private void configurarJanelaDependente(Frame owner) {
        setAutoRequestFocus(true);
        setAlwaysOnTop(true);

        if (owner == null) {
```

```
        setLocationByPlatform(true);  
        return;  
    }
```

```
    posicionarRelativaAoOwner(owner);  
}
```

```
private void posicionarRelativaAoOwner(Frame owner) {  
    int x = owner.getX() + (owner.getWidth() - getWidth()) / 2;  
    int y = owner.getY() + (owner.getHeight() - getHeight()) / 2;  
    setLocation(x, y);  
}
```

```
private Seguradora localizarSeguradora(String nome) {  
    if (nome == null) return null;  
    for (Seguradora visao : seguradoras_cadastradas) {  
        if (nome.equals(visao.getNome())) return visao;  
    }  
    return null;  
}
```

```
private void atualizarSeguradorasCadastradas() {  
    atualizarSeguradorasCadastradas(null);  
}
```

```
private void atualizarSeguradorasCadastradas(String nome_selecionado) {  
    seguradoras_cadastradas = Seguradora.getVisoes();  
    seguradoras_cadastradasComboBox.setModel(  
        new DefaultComboBoxModel(seguradoras_cadastradas)  
    );  
}
```

```
    Seguradora visao_selecionada = localizarSeguradora(nome_selecionado);  
    if (visao_selecionada != null) {  
        seguradoras_cadastradasComboBox.setSelectedItem(visao_selecionada);  
    }  
}
```

```
private void informarErro(String mensagem) {  
    JOptionPane.showMessageDialog(  
        this,  
        mensagem,  
        "Erro",  
        JOptionPane.ERROR_MESSAGE  
    );  
}
```

```
private Seguradora.FormaPagamentoPreferencial  
obterFormaPagamentoPreferencialSelecionada() {  
    Object selecionado = formaPagamentoPreferencialComboBox.getSelectedItem();  
    if (selecionado == null) {  
        return Seguradora.FormaPagamentoPreferencial.BOLETO;  
    }  
  
    return Seguradora.FormaPagamentoPreferencial.fromTexto(selecionado.toString());  
}
```

```
private Seguradora obterSeguradoraInformada() {  
  
    String nome = nomeTextField.getText();  
    if (nome.isEmpty()) return null;  
  
    String cidade = cidadeTextField.getText();  
    if (cidade.isEmpty()) cidade = null;  
  
    String cobertura_str = coberturaPercentualTextField.getText();  
    if (cobertura_str.isEmpty()) return null;  
  
    double cobertura_percentual;  
  
    try {  
        cobertura_percentual =  
            Double.parseDouble(cobertura_str.replace(",", "."));  
    } catch (NumberFormatException e) {  
        return null;  
    }
```

```
}
```

```
boolean possui_atendimento_24h =  
    possuiAtendimento24hCheckBox.isSelected();
```

```
return new Seguradora(  
    nome,  
    cidade,  
    cobertura_percentual,  
    possui_atendimento_24h,  
    obterFormaPagamentoPreferencialSelecionada()  
);  
}
```

```
private Seguradora getVisaoAlterada(String nome) {  
    for (Seguradora visao : seguradoras_cadastradas) {  
        if (visao.getNome().equals(nome)) return visao;  
    }  
    return null;  
}
```

```
@SuppressWarnings("unchecked")
```

```
// <editor-fold defaultstate="collapsed" desc="Generated Code">
```

```
private void initComponents() {  
    java.awt.GridBagConstraints gridBagConstraints;  
  
    comandosPanel = new javax.swing.JPanel();  
    inserirSeguradora = new javax.swing.JButton();  
    alterarSeguradora = new javax.swing.JButton();  
    consultarSeguradora = new javax.swing.JButton();  
    removerSeguradora = new javax.swing.JButton();  
    limparCampos = new javax.swing.JButton();  
    nomeLabel = new javax.swing.JLabel();  
    nomeTextField = new javax.swing.JTextField();  
    cidadeLabel = new javax.swing.JLabel();  
    cidadeTextField = new javax.swing.JTextField();  
    coberturaPercentualLabel = new javax.swing.JLabel();
```

```
coberturaPercentualTextField = new javax.swing.JTextField();
seguradorasCadastradasLabel = new javax.swing.JLabel();
seguradoras_cadastradasComboBox = new javax.swing.JComboBox();

setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
setTitle("Cadastrar Seguradoras");
setMinimumSize(new java.awt.Dimension(620, 360));
setPreferredSize(new java.awt.Dimension(700, 460));
getContentPane().setLayout(new java.awt.GridBagLayout());

inserirSeguradora.setText("Inserir");
inserirSeguradora.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        inserirSeguradora(evt);
    }
});
comandosPanel.add(inserirSeguradora);

alterarSeguradora.setText("Alterar");
alterarSeguradora.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        alterarSeguradora(evt);
    }
});
comandosPanel.add(alterarSeguradora);

consultarSeguradora.setText("Consultar");
consultarSeguradora.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        consultarSeguradora(evt);
    }
});
comandosPanel.add(consultarSeguradora);

removerSeguradora.setText("Remover");
removerSeguradora.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
```

```
        removerSeguradora(evt);
    }
});
comandosPanel.add(removerSeguradora);
```

```
limparCampos.setText("Limpar");
limparCampos.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        limparCampos(evt);
    }
});
comandosPanel.add(limparCampos);
```

```
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 5;
gridBagConstraints.insets = new java.awt.Insets(5, 5, 5, 5);
getContentPane().add(comandosPanel, gridBagConstraints);
```

```
nomeLabel.setText("Nome");
nomeLabel.setMaximumSize(null);
nomeLabel.setMinimumSize(null);
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 1;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 16, 10, 10);
getContentPane().add(nomeLabel, gridBagConstraints);
```

```
nomeTextField.setColumns(50);
nomeTextField.setPreferredSize(new java.awt.Dimension(240, 20));
nomeTextField.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        nomeTextFieldActionPerformed(evt);
    }
});
gridBagConstraints = new java.awt.GridBagConstraints();
```

```
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 1;
gridBagConstraints.ipadx = 80;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(nomeTextField, gridBagConstraints);
```

```
cidadeLabel.setText("Cidade");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 2;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 16, 10, 10);
getContentPane().add(cidadeLabel, gridBagConstraints);
```

```
cidadeTextField.setColumns(50);
cidadeTextField.setPreferredSize(new java.awt.Dimension(180, 20));
cidadeTextField.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        cidadeTextFieldActionPerformed(evt);
    }
});
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 2;
gridBagConstraints.ipadx = 20;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(cidadeTextField, gridBagConstraints);
```

```
coberturaPercentualLabel.setText("Cobertura Percentual");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 3;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 16, 10, 10);
getContentPane().add(coberturaPercentualLabel, gridBagConstraints);
```

```

coberturaPercentualTextField.setColumns(50);
coberturaPercentualTextField.setPreferredSize(new java.awt.Dimension(100, 20));
coberturaPercentualTextField.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        coberturaPercentualTextFieldActionPerformed(evt);
    }
});
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 3;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(coberturaPercentualTextField, gridBagConstraints);

seguradorasCadastradasLabel.setText("Seguradoras Cadastradas");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(0, 12, 0, 5);
getContentPane().add(seguradorasCadastradasLabel, gridBagConstraints);

seguradoras_cadastradasComboBox.setModel(new
DefaultComboBoxModel(seguradoras_cadastradas));
seguradoras_cadastradasComboBox.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        seguradoras_cadastradasComboBoxActionPerformed(evt);
    }
});
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 0;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(seguradoras_cadastradasComboBox, gridBagConstraints);

pack();
} // </editor-fold>

```

```
private void nomeTextFieldActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:  
}
```

```
private void cidadeTextFieldActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:  
}
```

```
private void coberturaPercentualTextFieldActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:  
}
```

```
private void alterarSeguradora(java.awt.event.ActionEvent evt) {  
    Seguradora seguradora =  
        obterSeguradoraInformada();
```

```
    String mensagem_erro = null;
```

```
    if (seguradora != null)  
        mensagem_erro =  
            controlador.alterarSeguradora(seguradora);
```

```
    else  
        mensagem_erro =  
            "Algum atributo da Seguradora não foi informado.";
```

```
    if (mensagem_erro == null) {
```

```
        Seguradora visao =  
            getVisaoAlterada(seguradora.getNome());
```

```
        if (visao != null) {
```

```
            visao.setCidade(  
                seguradora.getCidade()  
            );
```

```

        visao.setCoberturaPercentual(
            seguradora.getCoberturaPercentual()
        );

        visao.setPossuiAtendimento24h(
            seguradora.getPossuiAtendimento24h()
        );

        visao.setFormaPagamentoPreferencial(
            seguradora.getFormaPagamentoPreferencial()
        );

        seguradoras_cadastradasComboBox.updateUI();

        seguradoras_cadastradasComboBox
            .setSelectedItem(visao);
    }

} else {
    informarErro(mensagem_erro);
}
}

private void inserirSeguradora(java.awt.event.ActionEvent evt) {
    Seguradora seguradora =
        obterSeguradoraInformada();

    String mensagem_erro = null;

    if (seguradora != null)
        mensagem_erro =
            controlador.inserirSeguradora(seguradora);
    else
        mensagem_erro =
            "Algum atributo da Seguradora não foi informado";

    if (mensagem_erro == null) {

```

```

        Seguradora visao = seguradora.getVisao();
        atualizarSeguradorasCadastradas(visao.getNome());

    } else {
        informarErro(mensagem_erro);
    }
}

private void consultarSeguradora(java.awt.event.ActionEvent evt) {
    Seguradora visao =
        (Seguradora)
        seguradoras_cadastradasComboBox
            .getSelectedItem();

    Seguradora seguradora = null;

    String mensagem_erro = null;

    if (visao != null) {

        seguradora =
            Seguradora.buscarSeguradora(
                visao.getNome()
            );

        if (seguradora == null)
            mensagem_erro =
                "Seguradora não cadastrada";

    } else {
        mensagem_erro =
            "Nenhuma Seguradora selecionada";
    }

    if (mensagem_erro == null) {

```

```

nomeTextField.setText(
    seguradora.getNome()
);

String cidade =
    seguradora.getCidade();

if (cidade == null)
    cidade = "";

cidadeTextField.setText(cidade);

coberturaPercentualTextField.setText(
    String.valueOf(
        seguradora
            .getCoberturaPercentual()
    )
);

possuiAtendimento24hCheckBox.setSelected(
    seguradora.getPossuiAtendimento24h()
);

formaPagamentoPreferencialComboBox.setSelectedItem(
    seguradora.getFormaPagamentoPreferencial().toString()
);

} else {
    informarErro(mensagem_erro);
}
}

private void removerSeguradora(java.awt.event.ActionEvent evt) {
    Seguradora visao =
        (Seguradora)
        seguradoras_cadastradasComboBox
            .getSelectedItem();

```

```
String mensagem_erro = null;
```

```
if (visao != null)
```

```
    mensagem_erro =
```

```
        controlador.removerSeguradora(
```

```
            visao.getNome()
```

```
        );
```

```
else
```

```
    mensagem_erro =
```

```
        "Nenhuma Seguradora selecionada";
```

```
if (mensagem_erro == null) {
```

```
    atualizarSeguradorasCadastradas();
```

```
    limparCampos(evt);
```

```
} else {
```

```
    informarErro(mensagem_erro);
```

```
}
```

```
}
```

```
private void limparCampos(java.awt.event.ActionEvent evt) {
```

```
    nomeTextField.setText("");
```

```
    cidadeTextField.setText("");
```

```
    coberturaPercentualTextField.setText("");
```

```
    possuiAtendimento24hCheckBox.setSelected(false);
```

```
    formaPagamentoPreferencialComboBox.setSelectedItem("boleto");
```

```
}
```

```
private void seguradoras_cadastradasComboBoxActionPerformed(java.awt.event.ActionEvent  
evt) {
```

```
}
```

```
// Variables declaration - do not modify
private javax.swing.JButton alterarSeguradora;
private javax.swing.JLabel cidadeLabel;
private javax.swing.JTextField cidadeTextField;
private javax.swing.JLabel coberturaPercentualLabel;
private javax.swing.JTextField coberturaPercentualTextField;
private javax.swing.JPanel comandosPanel;
private javax.swing.JButton consultarSeguradora;
private javax.swing.JButton inserirSeguradora;
private javax.swing.JButton limparCampos;
private javax.swing.JLabel nomeLabel;
private javax.swing.JTextField nomeTextField;
private javax.swing.JButton removerSeguradora;
private javax.swing.JLabel seguradorasCadastradasLabel;
private javax.swing.JComboBox seguradoras_cadastradasComboBox;
// End of variables declaration
}
```

\$\$\$ src.interfaces.JanelaCadastroSinistros

```
package interfaces;

import controles.ControladorCadastroSinistros;
import entidades.Sinistro;
import java.awt.Frame;
import javax.swing.DefaultComboBoxModel;
import javax.swing.JOptionPane;

public class JanelaCadastroSinistros extends javax.swing.JFrame {

    ControladorCadastroSinistros controlador;
    Sinistro[] sinistros_cadastrados;

    public JanelaCadastroSinistros(ControladorCadastroSinistros controlador) {
        this(controlador, null);
    }

    public JanelaCadastroSinistros(ControladorCadastroSinistros controlador, Frame owner) {
        this.controlador = controlador;
        sinistros_cadastrados = Sinistro.getVisoes();
        initComponents();
        atualizarSinistrosCadastrados();
        setSize(new java.awt.Dimension(720, 470));
        configurarJanelaDependente(owner);
        limparCampos(null);
    }

    private void configurarJanelaDependente(Frame owner) {
        setAutoRequestFocus(true);
        setAlwaysOnTop(true);

        if (owner == null) {
            setLocationByPlatform(true);
            return;
        }
    }
}
```

```
int x = owner.getX() + (owner.getWidth() - getWidth()) / 2;
int y = owner.getY() + (owner.getHeight() - getHeight()) / 2;
setLocation(x, y);
}
```

```
private Sinistro localizarSinistro(int numero) {
    if (sinistros_cadastrados == null) return null;
    for (Sinistro visao : sinistros_cadastrados) {
        if (visao.getNumero() == numero) return visao;
    }
    return null;
}
```

```
private void atualizarSinistrosCadastrados() {
    atualizarSinistrosCadastrados(-1);
}
```

```
private void atualizarSinistrosCadastrados(int numero_selecionado) {
    sinistros_cadastrados = Sinistro.getVisoes();
    sinistros_cadastradosComboBox.setModel(new
DefaultComboBoxModel(sinistros_cadastrados));
```

```
    Sinistro visao_selecionada = localizarSinistro(numero_selecionado);
    if (visao_selecionada != null) {
        sinistros_cadastradosComboBox.setSelectedItem(visao_selecionada);
    }
}
```

```
private void informarErro(String mensagem) {
    JOptionPane.showMessageDialog(this, mensagem, "Erro",
JOptionPane.ERROR_MESSAGE);
}
```

```
private Sinistro obterSinistroInformado() {
    String numero_str = numeroTextField.getText().trim();
    if (numero_str.isEmpty()) return null;
```

```

int numero;
try {
    numero = Integer.parseInt(numero_str);
} catch (NumberFormatException excecao) {
    return null;
}

String cliente = clienteTextField.getText().trim();
if (cliente.isEmpty()) return null;

String telefone = telefoneTextField.getText().trim();
if (telefone.isEmpty()) return null;

Sinistro.GrauMonta grau_monta =
    (Sinistro.GrauMonta) grauMontaComboBox.getSelectedItem();
if (grau_monta == null) return null;

boolean perda_total = perdaTotalCheckBox.isSelected();

return new Sinistro(numero, cliente, telefone, grau_monta, perda_total);
}

private Sinistro getVisaoAlterada(int numero) {
    for (Sinistro visao : sinistros_cadastrados) {
        if (visao.getNumero() == numero) return visao;
    }
    return null;
}

@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {
    java.awt.GridBagConstraints gridBagConstraints;

    comandosPanel = new javax.swing.JPanel();
    inserirSinistro = new javax.swing.JButton();

```

```
alterarSinistro = new javax.swing.JButton();
consultarSinistro = new javax.swing.JButton();
removerSinistro = new javax.swing.JButton();
limparCampos = new javax.swing.JButton();
sinistrosCadastradosLabel = new javax.swing.JLabel();
sinistros_cadastradosComboBox = new javax.swing.JComboBox();
numeroLabel = new javax.swing.JLabel();
numeroTextField = new javax.swing.JTextField();
clienteLabel = new javax.swing.JLabel();
clienteTextField = new javax.swing.JTextField();
telefoneLabel = new javax.swing.JLabel();
telefoneTextField = new javax.swing.JTextField();
grauMontaLabel = new javax.swing.JLabel();
grauMontaComboBox = new javax.swing.JComboBox();
perdaTotalLabel = new javax.swing.JLabel();
perdaTotalCheckBox = new javax.swing.JCheckBox();

setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
setTitle("Cadastrar Sinistros");
setMinimumSize(new java.awt.Dimension(680, 420));
setPreferredSize(new java.awt.Dimension(720, 470));
getContentPane().setLayout(new java.awt.GridBagLayout());

inserirSinistro.setText("Inserir");
inserirSinistro.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        inserirSinistro(evt);
    }
});
comandosPanel.add(inserirSinistro);

alterarSinistro.setText("Alterar");
alterarSinistro.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        alterarSinistro(evt);
    }
});
```

```
comandosPanel.add(alterarSinistro);
```

```
consultarSinistro.setText("Consultar");
```

```
consultarSinistro.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        consultarSinistro(evt);  
    }  
});
```

```
comandosPanel.add(consultarSinistro);
```

```
removerSinistro.setText("Remover");
```

```
removerSinistro.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        removerSinistro(evt);  
    }  
});
```

```
comandosPanel.add(removerSinistro);
```

```
limparCampos.setText("Limpar");
```

```
limparCampos.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        limparCampos(evt);  
    }  
});
```

```
comandosPanel.add(limparCampos);
```

```
gridBagConstraints = new java.awt.GridBagConstraints();
```

```
gridBagConstraints.gridx = 1;
```

```
gridBagConstraints.gridy = 6;
```

```
gridBagConstraints.insets = new java.awt.Insets(5, 5, 5, 5);
```

```
getContentPane().add(comandosPanel, gridBagConstraints);
```

```
sinistrosCadastradosLabel.setText("Sinistros Cadastrados");
```

```
gridBagConstraints = new java.awt.GridBagConstraints();
```

```
gridBagConstraints.gridx = 0;
```

```
gridBagConstraints.gridy = 0;
```

```
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
```

```
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(sinistrosCadastradosLabel, gridBagConstraintss);
```

```
sinistros_cadastradosComboBox.setModel(new  
DefaultComboBoxModel(sinistros_cadastrados));  
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 1;  
gridBagConstraints.gridy = 0;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;  
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(sinistros_cadastradosComboBox, gridBagConstraintss);
```

```
numeroLabel.setText("Número");  
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 0;  
gridBagConstraints.gridy = 1;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;  
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(numeroLabel, gridBagConstraintss);
```

```
numeroTextField.setColumns(10);  
numeroTextField.setPreferredSize(new java.awt.Dimension(100, 20));  
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 1;  
gridBagConstraints.gridy = 1;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;  
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(numeroTextField, gridBagConstraintss);
```

```
clienteLabel.setText("Cliente");  
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 0;  
gridBagConstraints.gridy = 2;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;  
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(clienteLabel, gridBagConstraintss);
```

```
clienteTextField.setColumns(24);
clienteTextField.setPreferredSize(new java.awt.Dimension(240, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 2;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(clienteTextField, gridBagConstraints);
```

```
telefoneLabel.setText("Telefone");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 3;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(telefoneLabel, gridBagConstraints);
```

```
telefoneTextField.setColumns(16);
telefoneTextField.setPreferredSize(new java.awt.Dimension(160, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 3;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(telefoneTextField, gridBagConstraints);
```

```
grauMontaLabel.setText("Grau de Monta");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 4;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(grauMontaLabel, gridBagConstraints);
```

```
grauMontaComboBox.setModel(new
DefaultComboBoxModel(Sinistro.GrauMonta.values()));
gridBagConstraints = new java.awt.GridBagConstraints();
```

```

gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 4;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(grauMontaComboBox, gridBagConstraints);

perdaTotalLabel.setText("Perda Total");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 5;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(perdaTotalLabel, gridBagConstraints);

perdaTotalCheckBox.setText("Sim");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 5;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(perdaTotalCheckBox, gridBagConstraints);

pack();
} // </editor-fold>

private void limparCampos(java.awt.event.ActionEvent evt) {
    numeroTextField.setText("");
    numeroTextField.setEditable(true);
    clienteTextField.setText("");
    telefoneTextField.setText("");
    grauMontaComboBox.setSelectedIndex(0);
    perdaTotalCheckBox.setSelected(false);
    sinistros_cadastradosComboBox.setSelectedIndex(-1);
}

private void consultarSinistro(java.awt.event.ActionEvent evt) {
    Sinistro visao = (Sinistro) sinistros_cadastradosComboBox.getSelectedItem();

```

```

String mensagem_erro = null;
if (visao == null) {
    mensagem_erro = "Nenhum Sinistro selecionado";
} else {
    Sinistro sinistro = Sinistro.buscarSinistro(visao.getNumero());
    if (sinistro == null) {
        mensagem_erro = "Sinistro não cadastrado";
    } else {
        numeroTextField.setText(String.valueOf(sinistro.getNumero()));
        numeroTextField.setEditable(false);
        clienteTextField.setText(sinistro.getCliente());
        telefoneTextField.setText(sinistro.getTelefone());
        grauMontaComboBox.setSelectedItem(sinistro.getGrauMonta());
        perdaTotalCheckBox.setSelected(sinistro.getPerdaTotal());
    }
}

if (mensagem_erro != null) informarErro(mensagem_erro);
}

private void inserirSinistro(java.awt.event.ActionEvent evt) {
    Sinistro sinistro = obterSinistroInformado();
    String mensagem_erro = null;

    if (sinistro != null) {
        mensagem_erro = controlador.inserirSinistro(sinistro);
    } else {
        mensagem_erro = "Algum atributo do Sinistro não foi informado";
    }

    if (mensagem_erro == null) {
        atualizarSinistrosCadastrados(sinistro.getNumero());
        numeroTextField.setText(String.valueOf(sinistro.getNumero()));
        numeroTextField.setEditable(false);
        sinistros_cadastradosComboBox.setSelectedItem(getVisaoAlterada(sinistro.getNumero()));
    } else {
        informarErro(mensagem_erro);
    }
}

```

```
}  
}
```

```
private void alterarSinistro(java.awt.event.ActionEvent evt) {  
    Sinistro sinistro = obterSinistroInformado();  
    String mensagem_erro = null;  
  
    if (sinistro != null) {  
        mensagem_erro = controlador.alterarSinistro(sinistro);  
    } else {  
        mensagem_erro = "Algum atributo do Sinistro não foi informado";  
    }  
  
    if (mensagem_erro == null) {  
        atualizarSinistrosCadastrados(sinistro.getNumero());  
        sinistros_cadastradosComboBox.setSelectedItem(getVisaoAlterada(sinistro.getNumero()));  
    } else {  
        informarErro(mensagem_erro);  
    }  
}
```

```
private void removerSinistro(java.awt.event.ActionEvent evt) {  
    Sinistro visao = (Sinistro) sinistros_cadastradosComboBox.getSelectedItem();  
    String mensagem_erro = null;  
  
    if (visao != null) {  
        mensagem_erro = controlador.removerSinistro(visao.getNumero());  
    } else {  
        mensagem_erro = "Nenhum Sinistro selecionado";  
    }  
  
    if (mensagem_erro == null) {  
        atualizarSinistrosCadastrados();  
        limparCampos(evt);  
    } else {  
        informarErro(mensagem_erro);  
    }  
}
```

```
}
```

```
// Variables declaration - do not modify
```

```
private javax.swing.JButton alterarSinistro;
```

```
private javax.swing.JButton consultarSinistro;
```

```
private javax.swing.JTextField clienteTextField;
```

```
private javax.swing.JLabel clienteLabel;
```

```
private javax.swing.JPanel comandosPanel;
```

```
private javax.swing.JButton inserirSinistro;
```

```
private javax.swing.JButton limparCampos;
```

```
private javax.swing.JComboBox grauMontaComboBox;
```

```
private javax.swing.JLabel grauMontaLabel;
```

```
private javax.swing.JLabel numeroLabel;
```

```
private javax.swing.JTextField numeroTextField;
```

```
private javax.swing.JCheckBox perdaTotalCheckBox;
```

```
private javax.swing.JLabel perdaTotalLabel;
```

```
private javax.swing.JButton removerSinistro;
```

```
private javax.swing.JLabel sinistrosCadastradosLabel;
```

```
private javax.swing.JComboBox sinistros_cadastradosComboBox;
```

```
private javax.swing.JLabel telefoneLabel;
```

```
private javax.swing.JTextField telefoneTextField;
```

```
// End of variables declaration
```

```
}
```

\$\$\$ src.interfaces.JanelaSistema

```
package interfaces;

import controles.ControladorCadastroSeguradoras;
import controles.ControladorCadastroPecas;
import controles.ControladorCadastroSinistros;
import javax.swing.JOptionPane;
import persistência.BD;

public class JanelaSistema extends javax.swing.JFrame {

    public JanelaSistema() {
        BD.criaConexao();
        initComponents();
    }

    @SuppressWarnings("unchecked")

    private void informarServiçoIndisponível() {
        JOptionPane.showMessageDialog(
            this,
            "Serviço Indisponível",
            "Informação",
            JOptionPane.INFORMATION_MESSAGE
        );
    }

    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        seguradoras_orçamentoMenuBar = new javax.swing.JMenuBar();
        sinistroMenu = new javax.swing.JMenu();
        cadastrar_sinistroMenuItem = new javax.swing.JMenuItem();
        seguradoraMenu = new javax.swing.JMenu();
        cadastrar_seguradoraItemMenu = new javax.swing.JMenuItem();
        peçaMenu = new javax.swing.JMenu();
```

```

cadastrar_peçaMenuItem = new javax.swing.JMenuItem();
orçamentoMenu = new javax.swing.JMenu();
cadastrar_orçamentoMenuItem = new javax.swing.JMenuItem();
pesquisar_orçamentoMenuItem = new javax.swing.JMenuItem();

setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
setTitle("Cadastro de Seguradoras e Peças");
setAlwaysOnTop(true);
addWindowListener(new java.awt.event.WindowAdapter() {
    public void windowClosed(java.awt.event.WindowEvent evt) {
        terminarSistema(evt);
    }
});

sinistroMenu.setText("Sinistro");

cadastrar_sinistroMenuItem.setText("Cadastrar");
cadastrar_sinistroMenuItem.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        cadastrarSinistro(evt);
    }
});
sinistroMenu.add(cadastrar_sinistroMenuItem);

seguradoras_orçamentoMenuBar.add(sinistroMenu);

seguradoraMenu.setText("Seguradora");

cadastrar_seguradoraItemMenu.setText("Cadastrar");
cadastrar_seguradoraItemMenu.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        cadastrarSeguradora(evt);
    }
});
seguradoraMenu.add(cadastrar_seguradoraItemMenu);

seguradoras_orçamentoMenuBar.add(seguradoraMenu);

```

```
peçaMenu.setText("Peça");
```

```
cadastrar_peçaMenuItem.setText("Cadastrar");
```

```
cadastrar_peçaMenuItem.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        cadastrarPeça(evt);  
    }  
});
```

```
peçaMenu.add(cadastrar_peçaMenuItem);
```

```
seguradoras_orçamentoMenuBar.add(peçaMenu);
```

```
orçamentoMenu.setText("Orçamento");
```

```
cadastrar_orçamentoMenuItem.setText("Cadastrar");
```

```
cadastrar_orçamentoMenuItem.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        cadastrarOrçamentos(evt);  
    }  
});
```

```
orçamentoMenu.add(cadastrar_orçamentoMenuItem);
```

```
pesquisar_orçamentoMenuItem.setText("Pesquisar");
```

```
pesquisar_orçamentoMenuItem.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        pesquisarOrçamentos(evt);  
    }  
});
```

```
orçamentoMenu.add(pesquisar_orçamentoMenuItem);
```

```
seguradoras_orçamentoMenuBar.add(orçamentoMenu);
```

```
setJMenuBar(seguradoras_orçamentoMenuBar);
```

```
javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());  
getContentPane().setLayout(layout);
```

```
layout.setHorizontalGroup(  
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)  
    .addGap(0, 400, Short.MAX_VALUE)  
);  
layout.setVerticalGroup(  
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)  
    .addGap(0, 278, Short.MAX_VALUE)  
);  
  
pack();  
} // </editor-fold>
```

```
private void cadastrarSeguradora(java.awt.event.ActionEvent evt) {  
    new ControladorCadastroSeguradoras(this);  
}
```

```
private void terminarSistema(java.awt.event.WindowEvent evt) {  
    BD.fechaConexao();  
    System.exit(0);  
}
```

```
private void pesquisarOrçamentos(java.awt.event.ActionEvent evt) {  
    informarServiçoIndisponível();  
}
```

```
private void cadastrarPeça(java.awt.event.ActionEvent evt) {  
    new ControladorCadastroPecas(this);  
}
```

```
private void cadastrarOrçamentos(java.awt.event.ActionEvent evt) {  
    informarServiçoIndisponível();  
}
```

```
private void cadastrarSinistro(java.awt.event.ActionEvent evt) {  
    new ControladorCadastroSinistros();  
}
```

```
public static void main(String args[]) {  
    java.awt.EventQueue.invokeLater(new Runnable() {  
        public void run() {  
            new JanelaSistema().setVisible(true);  
        }  
    });  
}
```

```
// Variables declaration - do not modify
```

```
private javax.swing.JMenuItem cadastrar_orçamentoMenuItem;  
private javax.swing.JMenuItem cadastrar_peçaMenuItem;  
private javax.swing.JMenuItem cadastrar_seguradoraItemMenu;  
private javax.swing.JMenuItem cadastrar_sinistroMenuItem;  
private javax.swing.JMenu orçamentoMenu;  
private javax.swing.JMenuItem pesquisar_orçamentoMenuItem;  
private javax.swing.JMenu peçaMenu;  
private javax.swing.JMenu seguradoraMenu;  
private javax.swing.JMenuBar seguradoras_orçamentoMenuBar;  
private javax.swing.JMenu sinistroMenu;
```

```
// End of variables declaration
```

```
}
```

\$\$\$ src.persistência.BD

```
package persistência;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class BD {

    static final String URL_BD = "jdbc:mysql://localhost/banco";
    static final String USUARIO = "root";
    static final String SENHA = "admin";
    public static Connection conexao = null;

    public static void criaConexao() {
        try {
            conexao = DriverManager.getConnection(URL_BD, USUARIO, SENHA);
        } catch (SQLException excecao_sql) {
            excecao_sql.printStackTrace();
        }
    }

    public static void fechaConexao() {
        try {
            if (conexao != null) {
                conexao.close();
            }
        } catch (SQLException excecao_sql) {
            excecao_sql.printStackTrace();
        }
    }
}
```